WORKSHOP

Computer Vision

with Python & OpenCV

Python 3.9+ | OpenCV | YOLO | MediaPipe

What You Will Learn

Images as numbers
How computers store and read visual data

Core OpenCV Operations
Load, process, filter and display images

Color & Edge Detection
HSV masking, Canny edges, contours

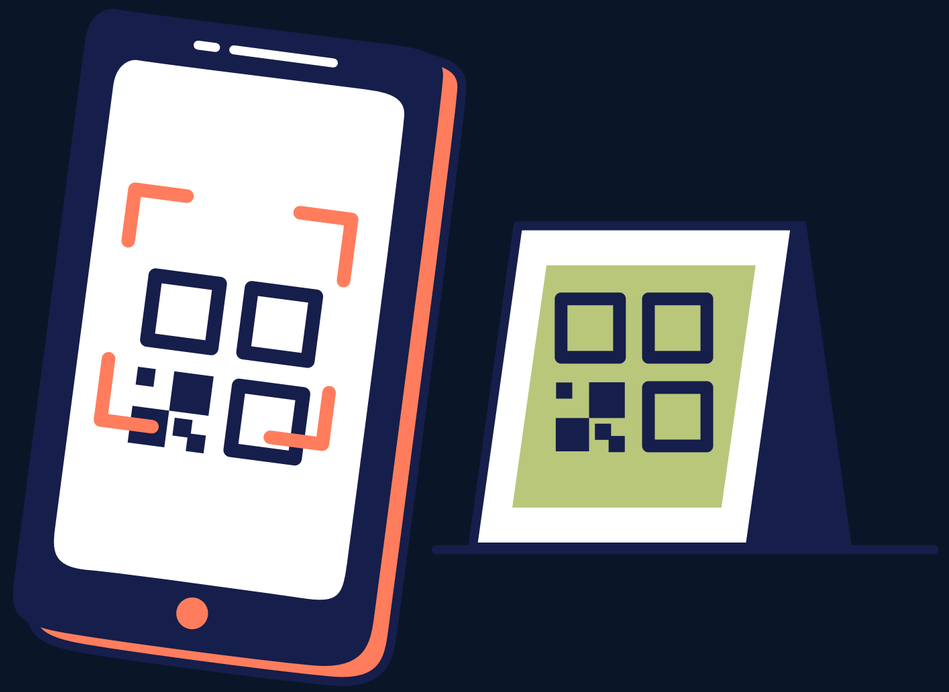
Face Detection
Haar Cascades with live webcam

YOLO AI Detection
Detect 80+ object classes in real time

11 Real Projects
From beginner to advanced in one session

Duration: 2.5 Hours · Level: Beginner → Advanced · Tools: PyCharm + Python

Before We Start...



INTRODUCTION

What is Computer Vision?

Teaching machines to interpret and understand the visual world

Humans see using eyes & brain.
Computers
see using cameras & math.

THE PIPELINE

Camera › OpenCV › **AI Model** › Output

REAL-WORLD APPLICATIONS

Face Unlock
Facial detection & recognition

Self-Driving Cars
Object & lane detection

Security Cameras
Motion & person detection

Medical Imaging
Tumour analysis in X-rays

AR Filters
Face landmark tracking

Robotics
Object avoidance & grasping

FUNDAMENTALS

Images are Just Numbers

Every pixel is a value between 0 (black) and 255 (white)

GRAYSCALE IMAGE

0	30	80	120	150
50	100	160	200	220
20	70	130	180	240
10	60	110	170	210
0	40	90	140	255

Each number = brightness of one pixel

COLOR IMAGE (BGR)

`[255, 0, 0]` Blue

`[0, 255, 0]` Green

`[0, 0, 255]` Red

`[255,255,0]` Cyan

`[255,0,255]` Magenta

IMAGE SHAPE

`img.shape → ( 720, 1280, 3 )`

720 = Height | 1280 = Width | 3 = Channels

IMPORTANT

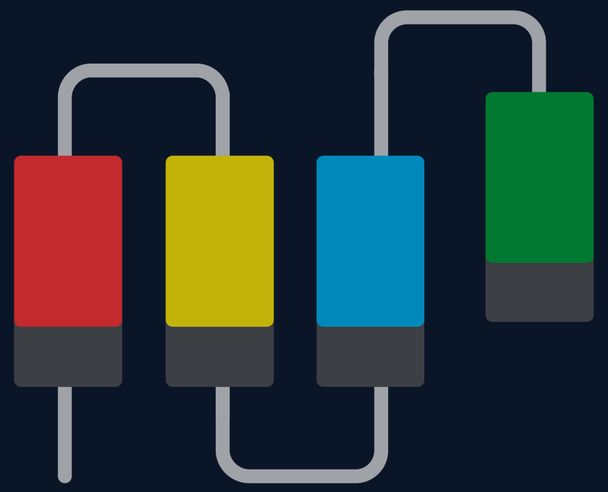
img.shape → (height, width, ch)
cv2.resize → (width, height)

They are opposite — height first vs width first!

⚠ OpenCV uses BGR — NOT RGB!
Most other tools use RGB order.

ARCHITECTURE

OpenCV Processing Pipeline



Every CV program follows this same sequence of operations

01 📷 **Image Acquisition**

Camera, video file, or image on disk

`cv2.VideoCapture(0)`

02 🔢 **Representation**

NumPy array — shape (H × W × 3)

`img.shape → (720, 1280, 3)`

03 🧹 **Preprocessing**

Resize, convert color, blur, denoise

`cvtColor, GaussianBlur, resize`

04 🔍 **Feature Detection**

Find edges, contours, keypoints

`Canny, findContours`

05 🎯 **Object Detection**

Haar Cascades, YOLO neural network

`detectMultiScale, YOLO()`

06 🖥️ **Visualization**

Draw boxes, text, display result

`rectangle, putText, imshow`

SETUP

Environment Setup

Get everything installed before writing your first line of code

1 **Install Python 3.9 – 3.11**

Download from python.org · Avoid Python 3.12+ (MediaPipe not supported yet)

```
python --version
```

2 **Install PyCharm Community Edition**

Free IDE — use the built-in terminal for all pip installs

```
jetbrains.com/pycharm → New Project → Pure Python
```

3 **Install All Libraries**

Run this in PyCharm's terminal — ensures the correct Python environment

```
pip install opencv-python numpy ultralytics mediapipe pyautogui cvzone pynput
```

4 **Verify Installation**

No errors = ready to go!

```
import cv2; print(cv2.__version__) → should print 4.x.x
```

CODE

Core OpenCV Operations

The essential functions you will use in every project

first_program.py

```python
import cv2

# Load image from disk
img = cv2.imread("sample.jpg")

# Check dimensions
print("Shape:", img.shape)
# → (720, 1280, 3)

# Display result
cv2.imshow("Window", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

FUNCTION REFERENCE

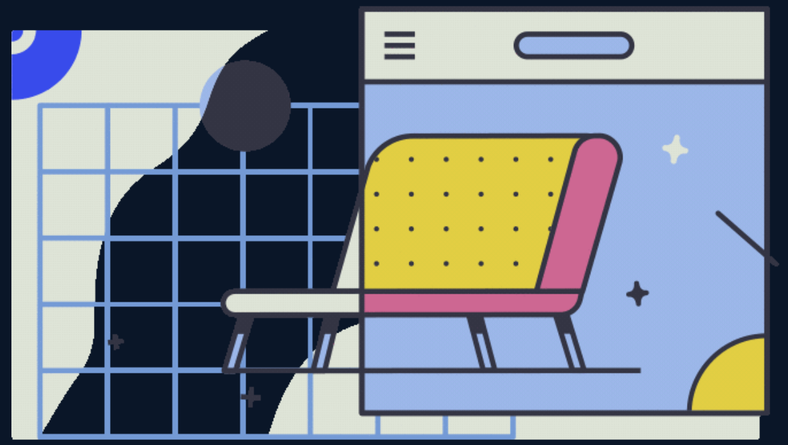
`cv2.imread()`	Load image file → NumPy BGR array
`cv2.imshow()`	Display image in a named window
`cv2.waitKey(0)`	Wait forever for keypress
`cv2.waitKey(1)`	Wait 1ms — use this in video loops
`cv2.imwrite()`	Save processed image to disk
`cv2.resize()`	Resize: takes (width, height)
`cv2.cvtColor()`	Convert between color spaces

⚠ Never use waitKey(0) inside a video loop — it will freeze on the first frame.

TECHNIQUES

Color & Edge Detection

HSV masking for color tracking · Canny for object boundaries



EDGE DETECTION (CANNY)

```
gray = cv2.cvtColor(frame,cv2.COLOR_BGR2GRAY)

edges = cv2.Canny(gray, 100, 200)

cv2.imshow("Sketch Cam", edges)
```

COLOR DETECTION (HSV)

```
hsv = cv2.cvtColor(frame,cv2.COLOR_BGR2HSV)

lower = np.array([100, 150, 0])
upper = np.array([140, 255, 255])

mask = cv2.inRange(hsv, lower, upper)
```

Analogy: Like tracing the outline of every object with a pencil. Lower thresholds → more edges detected.

COMMON HSV RANGES

Blue	[100, 150, 0]	→ [140, 255, 255]
Green	[40, 50, 50]	→ [80, 255, 255]
Red	[0, 120, 70]	→ [10, 255, 255]
Yellow	[20, 100, 100]	→ [30, 255, 255]

FACE DETECTION

Face Detection with Haar Cascades

Pretrained models shipped with OpenCV — no training required

face_detection.py

```python
import cv2

# Load pretrained Haar Cascade model
face = cv2.CascadeClassifier(
    cv2.data.haarcascades
    + "haarcascade_frontalface_default.xml")

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    gray = cv2.cvtColor(frame,
              cv2.COLOR_BGR2GRAY)
    faces = face.detectMultiScale(gray,1.3,5)
    for (x,y,w,h) in faces:
        cv2.rectangle(frame,(x,y),(x+w,y+h),
                      (0,255,0),2)
```

CascadeClassifier

Loads the pre-trained XML model file. OpenCV ships with face, eye, and smile cascades built in.

detectMultiScale

Scans image at multiple scales. scaleFactor=1.3 scales by 30% each pass. minNeighbors=5 reduces false positives.

Return Value

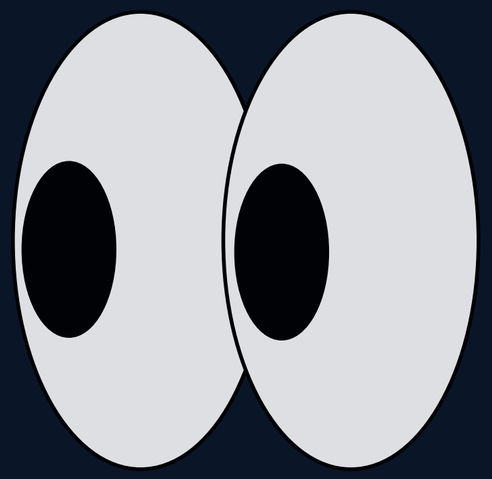
Returns list of (x, y, w, h) tuples — the top-left corner and dimensions of each detected face.

Tip: Also try haarcascade_eye.xml to detect eyes!

DEEP LEARNING

YOLO — You Only Look Once

Real-time AI object detection in a single neural network pass



CLASSICAL COMPUTER VISION

Image → Hand-crafted Features → Rule-based Detection

YOLO — DEEP LEARNING

Image → **Neural Network** → 80+ Classes Detected

Each detection returns:

- Class label (e.g. person, car, phone)
- Bounding box (x, y, width, height)
- Confidence (0.0 – 1.0)

MODEL COMPARISON

Model	Speed	mAP	Best For
yolov8n.pt	Fastest	37.3	Workshop / low-end PC
yolov8s.pt	Fast	44.9	Everyday use
yolov8m.pt	Moderate	50.2	Production / accuracy

yolo_webcam.py

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")    cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    results = model(frame)
    out = results[0].plot()
    cv2.imshow("YOLO", out)
```

PROJECTS

11 Hands-On Projects

Progressive difficulty — build confidence with every project

01
Edge Detection Camera
Canny algorithm

02
Blue Color Tracker
HSV masking

03
Face Detection
Haar Cascade

04
Motion Detection
Frame differencing

05
Portrait Mode Blur
Selective masking

06
Invisible Cloak ✨
Bitwise operations

07
YOLO Real-Time Detection
Deep learning

08
Object Counter
Box counting

09
Eye Controlled Mouse
MediaPipe landmarks

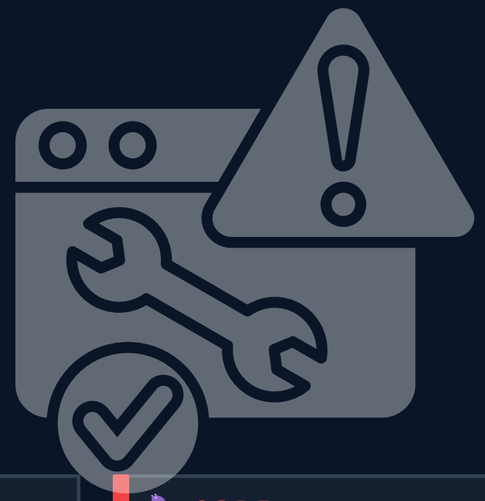
Mouse via Eye Tracking
Iris + PyAutoGUI

Virtual Keyboard
Hand gesture control

REFERENCE

Troubleshooting Quick Reference

Common issues every student encounters — and exactly how to fix them



📷 CAMERA

Webcam not opening

Change VideoCapture(0) → try (1) or (2)

📷 CAMERA

First frame is black

Add time.sleep(1) before the while loop

📷 CAMERA

Webcam LED stays on

Always call cap.release() at the end

📦 INSTALL

pip installs wrong Python

Use pip3 instead of pip

📦 INSTALL

MediaPipe fails

Only works on Python 3.9–3.11 (not 3.12+

📦 INSTALL

cvzone version error

Run pip install cvzone==1.5.6 exactly

🐛 CODE

Freezes on first frame

Using waitKey(0) in loop — change to wait

🐛 CODE

Image stretched / squished

resize() takes (width, height) — width FI

🐛 CODE

Wrong colors or no detection

OpenCV=BGR · MediaPipe needs RGB — use

🤖 YOLO

First run hangs

Downloading model (~6 MB) — wait, only on

🤖 YOLO

Very slow / low FPS

Use yolov8n.pt (nano) — the fastest versi

👁 EYE

Mouse stuck in corner

Add pyautogui.FAILSAFE = False at top

You can now make computers

see.

OpenCV Computer Vision Workshop

TOOLS USED IN THIS WORKSHOP

Python 3.9–3.11	PyCharm IDE
OpenCV 4.x	NumPy
Ultralytics YOLO	MediaPipe
cvzone	pynput

LEARNING OUTCOMES

- ✓ What Computer Vision is and real-world uses
- ✓ How images are stored as numerical arrays
- ✓ Core OpenCV image processing operations
- ✓ Live webcam capture and video processing
- ✓ Color detection using HSV color space
- ✓ Face detection with Haar Cascade models
- ✓ Difference between classical CV and AI
- ✓ YOLO real-time object detection (80+ classes)
- ✓ 11 complete hands-on projects
- ✓ Troubleshooting common OpenCV errors